Formatting in interpolated stirng expressions $"{}"

Syntax Structure

The basic syntax for applying format specifiers inside an interpolated string is:

```
$"expression {expression,alignment:formatString}"
```

- `expression` : The variable or calculation.
- `alignment` *(optional)*: A signed integer. Positive = right-aligned, Negative = left-aligned (padded with spaces).
- `formatString` : The standard or custom format pattern (preceded by a colon `:`).

Common Standard Format Specifiers

Category	Specifier	Description	Example Code	Output
Numeric	`N2`	Number with commas & 2 decimal places	`$"Result: {val:N2}"`	`Result: 1,234.57`
	`F4`	Fixed-point with 4 decimal places	`$"Result: {val:F4}"`	`Result: 1234.5678`
	`D6`	Integer padding with leading zeros (ints only)	`int id = 42; $"ID: {id:D6}"`	`ID: 000042`
Currency	`C` or `C2`	Currency format based on current culture	`decimal price = 49.9m; $"Price: {price:C}"`	`$49.90` (locale-dependent)
Percent	`P2`	Percentage (multiplies by 100 & adds `%`)	`double rate = 0.854; $"Rate: {rate:P2}"`	`Rate: 85.40%`
Hexadecimal	`X` or `x`	Hexadecimal representation (integers only)	`int color = 255; $"Hex: {color:X}"`	`Hex: FF`

Date and Time Formatting

Format Character	Description	Example (DateTime dt = new DateTime(2026, 10, 23, 14, 5, 9);)	Output
`d`	Short date	`$"Date: {dt:d}"`	`10/23/2026`
`D`	Long date	`$"Date: {dt:D}"`	`Friday, October 23, 2026`
`t`	Short time	`$"Time: {dt:t}"`	`2:05 PM`
`T`	Long time	`$"Time: {dt:T}"`	`2:05:09 PM`
`yyyy-MM-dd`	Custom ISO format	`$"ISO: {dt:yyyy-MM-dd}"`	`2026-10-23`
`hh:mm:ss tt`	Custom 12-hour format	`$"Custom: {dt:hh:mm:ss tt}"`	`02:05:09 PM`

Alignment & Padding

Control column width and spacing inside the interpolation slot using a comma followed by a number.

```
string item = "Apple";
decimal price = 1.25m;

// Positive number = Right-aligned in a 10-character wide field
// Negative number = Left-aligned in a 10-character wide field
Console.WriteLine($"|{item,-10}|{price,10:C}|");
```

Output: Plaintext

```
|Apple     |     $1.25|
```

Escaping Special Characters (`{` , `}` , and Colons)

Because curly braces `{` and `}` have structural meaning in interpolated strings, you must escape them by doubling them:

- **Literal Braces**: Double them up (`{` becomes `{{` , `}` becomes `}}`). `C# int val = 5; string s = $"{{ Value is: {val} }}"; // Output: "{ Value is: 5 }"`